

Parte de clasificación (vowel)

Andrei Kostin

Descargamos las bibliotecas necesarias:

```
library(readr)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v purrr      1.1.0
## v forcats    1.0.1      v stringr   1.5.2
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(dplyr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(caret)
```

```
##           : lattice
##
##           : 'caret'
##
##           'package:purrr':
##
## lift
```

```
library(class)
library(ggthemes)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(MASS)
```

```
##
##           : 'MASS'
##
##           'package:dplyr':
##
## select
```

Leemos el dataset objetivo y nombramos las columnas. Las variables F0–F9 representan formantes acústicas y “Class” es la vocal a predecir.

```
df <- read.csv("D:/UGR/Introduction to DS/Tarea final/vowel/vowel.dat",
               comment.char="@", header = FALSE)

names(df) <- c("TT", "SpeakerNumber", "Sex",
               "F0", "F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "F9",
               "Class")

head(df)
```

```
##   TT SpeakerNumber Sex      F0      F1      F2      F3      F4      F5      F6      F7
## 1  0              0  0 -3.639 0.418 -0.670 1.779 -0.168 1.627 -0.388 0.529
## 2  0              0  0 -3.327 0.496 -0.694 1.365 -0.265 1.933 -0.363 0.510
## 3  0              0  0 -2.120 0.894 -1.576 0.147 -0.707 1.559 -0.579 0.676
## 4  0              0  0 -2.287 1.809 -1.498 1.012 -1.053 1.060 -0.567 0.235
## 5  0              0  0 -2.598 1.938 -0.846 1.062 -1.633 0.764 0.394 -0.150
## 6  0              0  0 -2.852 1.914 -0.755 0.825 -1.588 0.855 0.217 -0.246
##           F8      F9 Class
## 1 -0.874 -0.814      0
## 2 -0.621 -0.488      1
## 3 -0.809 -0.049      2
## 4 -0.091 -0.795      3
## 5 0.277 -0.396      4
## 6 0.238 -0.365      5
```

Cambio del tipo de las variables numéricas al tipo categórico:

```
df$TT <- factor(df$TT)
df$SpeakerNumber <- factor(df$SpeakerNumber)
df$Sex <- factor(df$Sex)
# Reagrupamos los niveles de Sex
df$Sex <- fct_collapse(df$Sex,
                       male = "0",
                       female = "1")
df$Class <- factor(df$Class)

glimpse(df) # la estructura de los datos
```

```
## Rows: 990
## Columns: 14
## $ TT          <fct> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ SpeakerNumber <fct> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ Sex          <fct> male, male, male, male, male, male, male, male, male, male, ma~
## $ F0           <dbl> -3.639, -3.327, -2.120, -2.287, -2.598, -2.852, -3.482, ~
## $ F1           <dbl> 0.418, 0.496, 0.894, 1.809, 1.938, 1.914, 2.524, 2.305, ~
## $ F2           <dbl> -0.670, -0.694, -1.576, -1.498, -0.846, -0.755, -0.433, ~
## $ F3           <dbl> 1.779, 1.365, 0.147, 1.012, 1.062, 0.825, 1.048, 1.771, ~
## $ F4           <dbl> -0.168, -0.265, -0.707, -1.053, -1.633, -1.588, -1.995, ~
## $ F5           <dbl> 1.627, 1.933, 1.559, 1.060, 0.764, 0.855, 0.902, 0.593, ~
## $ F6           <dbl> -0.388, -0.363, -0.579, -0.567, 0.394, 0.217, 0.322, -0.~
## $ F7           <dbl> 0.529, 0.510, 0.676, 0.235, -0.150, -0.246, 0.450, 0.992~
## $ F8           <dbl> -0.874, -0.621, -0.809, -0.091, 0.277, 0.238, 0.377, 0.5~
```

```
## $ F9          <dbl> -0.814, -0.488, -0.049, -0.795, -0.396, -0.365, -0.366, ~
## $ Class       <fct> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 0, 1, 2, 3, 4, 5, 6, 7~
```

1. Clasificación con el algoritmo k-NN

Normalización de los datos para el cálculo de distancias:

```
prepr <- preProcess(df[,4:13], method = "range")
df_scaled <- predict(prepr, df)
head(df_scaled)
```

```
##   TT SpeakerNumber Sex      F0      F1      F2      F3      F4
## 1  0              0 male 0.3681499 0.2665406 0.4637570 0.8420497 0.4949469
## 2  0              0 male 0.4412178 0.2788280 0.4576314 0.7326994 0.4704396
## 3  0              0 male 0.7238876 0.3415249 0.2325166 0.4109878 0.3587671
## 4  0              0 male 0.6847775 0.4856648 0.2524247 0.6394612 0.2713492
## 5  0              0 male 0.6119438 0.5059861 0.4188361 0.6526677 0.1248105
## 6  0              0 male 0.5524590 0.5022054 0.4420623 0.5900687 0.1361799
##              F5      F6      F7      F8      F9 Class
## 1 0.7786911 0.3908163 0.5468187 0.2529090 0.2815345      0
## 2 0.8754347 0.3993197 0.5411164 0.3394935 0.3875163      1
## 3 0.7571925 0.3258503 0.5909364 0.2751540 0.5302341      2
## 4 0.5994309 0.3299320 0.4585834 0.5208761 0.2877113      3
## 5 0.5058489 0.6568027 0.3430372 0.6468172 0.4174252      4
## 6 0.5346190 0.5965986 0.3142257 0.6334702 0.4275033      5
```

Definimos una función que entrena y evalúa k-NN según los archivos de train y test:

```
# sobre el parámetro return_data: si TRUE, devuelve también los datos escalados y las predicciones
# para hacer gráficos
run_knn_fold <- function(i, k = 5, tt = "test", return_data = FALSE) {
  file <- paste0("vowel-10-", i, "tra.dat")
  x_tra <- read.csv(file, comment.char="@", header=FALSE)
  file <- paste0("vowel-10-", i, "tst.dat")
  x_tst <- read.csv(file, comment.char="@", header=FALSE)
  names(x_tra) <- c("TT", "SpeakerNumber", "Sex",
                   "F0", "F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "F9",
                   "Class")
  names(x_tst) <- names(x_tra)

  x_tra$Class <- factor(x_tra$Class)
  x_tst$Class <- factor(x_tst$Class)

  # Normalizamos F0-F9
  prepr <- preProcess(x_tra[,4:13], method = "range")
  x_tra_scaled <- predict(prepr, x_tra[,4:13])
  x_tst_scaled <- predict(prepr, x_tst[,4:13])

  # Elegimos si queremos evaluar en train o en test
  if (tt == "train") {
    x_test <- x_tra_scaled
    y_test <- x_tra$Class
  }
```

```

else {
  x_test <- x_tst_scaled
  y_test <- x_tst$Class
}

# Aplicamos k-NN con los datos escalados de train
knn.pred <- knn(x_tra_scaled, x_test, cl = x_tra$Class, k = k)

if (return_data){
  # Devolvemos tanto los datos como las predicciones para visualización
  val <- cbind(x_test, Class = y_test)
  list(data = val, pred = knn.pred)
} else {
  # Calculamos y devolvemos el accuracy en el conjunto elegido
  t <- table(knn.pred, y_test)
  #print(t)
  sum(diag(t)) / length(y_test)
}
}

```

Por ejemplo, utilizamos esta función para calcular la precisión de k-NN sobre el test con $k = 5$ en el primer fold:

```
run_knn_fold(1, k = 5, tt = "test")
```

```
## [1] 0.959596
```

Calculamos la precisión media de k-NN ($k = 5$) en todos los folds:

```

acc_knn_test <- mean(sapply(1:10, run_knn_fold, tt = "test"))
acc_knn_train <- mean(sapply(1:10, run_knn_fold, tt = "train"))
res_knn <- data.frame(model = "k-NN",
                      acc_train = acc_knn_train,
                      acc_test = acc_knn_test)

```

```
res_knn
```

```

##   model acc_train acc_test
## 1  k-NN 0.9808081 0.9505051

```

El accuracy de train es muy alto (casi iguala 1), la precisión de test es un poco menos, pero también alta, lo que indica buen rendimiento con ligero sobreajuste.

Visualización de las predicciones de k-NN en el fold 1:

```

res_plot <- run_knn_fold(1, k = 5, tt = "test", return_data = TRUE)
val <- as.data.frame(res_plot$data)
knn.pred <- res_plot$pred

```

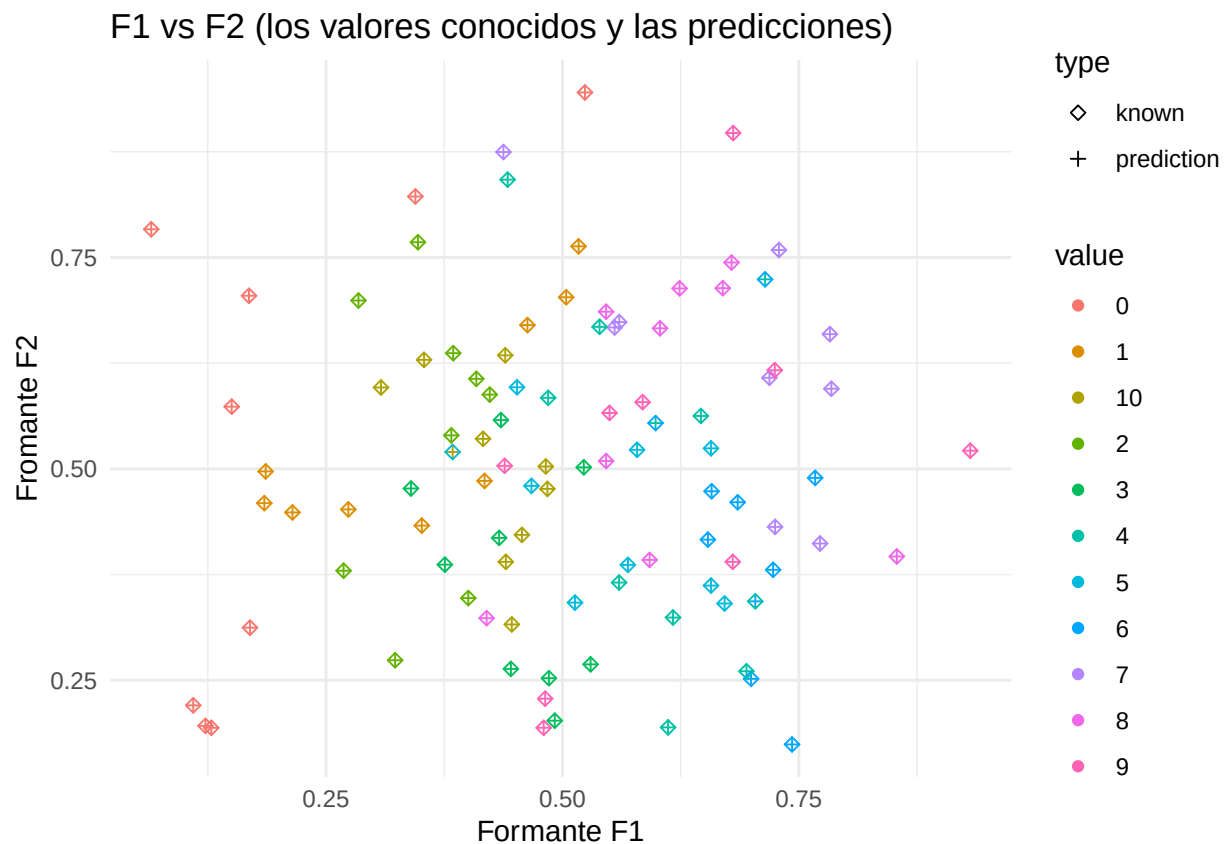
```

# Unificamos los niveles de las clases verdaderas y predichas para evitar problemas de factores
lev <- union(levels(factor(val$Class)), levels(factor(knn.pred)))
val$Class <- factor(val$Class, levels = lev)
knn.pred <- factor(knn.pred, levels = lev)

# Preparamos un dataframe long para graficar F1 vs F2
plot_data <- val %>%
  mutate(prediction = knn.pred) %>%
  rename(known = Class) %>%
  gather(type, value, prediction, known)

ggplot(plot_data) +
  geom_point(aes(x = F1, y = F2, shape = type, color = value)) +
  labs(x = "Formante F1", y = "Formante F2",
       title = "F1 vs F2 (los valores conocidos y las predicciones)") +
  scale_shape_manual(values = c(5, 3)) +
  theme_minimal()

```



Este gráfico muestra los puntos de test aplicados en el espacio F1 y F2, donde los rombos corresponden a clases verdaderas, y las cruces son predicciones k-NN. De acuerdo con el gráfico la mayoría de las cruces se colocan en rombos del mismo color, confirmando la alta precisión de la clasificación.

Demostración de la matriz de confusión:

```

cm <- confusionMatrix(knn.pred, val$Class)
cm

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction 0 1 2 3 4 5 6 7 8 9 10
##           0 9 0 0 0 0 0 0 0 0 0
##           1 0 9 0 0 0 0 0 0 0 0
##           2 0 0 9 0 0 0 0 0 0 0
##           3 0 0 0 9 0 0 0 0 0 0
##           4 0 0 0 0 8 0 2 0 0 0
##           5 0 0 0 0 0 8 0 0 0 0
##           6 0 0 0 0 1 0 7 0 0 0
##           7 0 0 0 0 0 0 0 9 0 0
##           8 0 0 0 0 0 0 0 0 9 0
##           9 0 0 0 0 0 0 0 0 0 9
##          10 0 0 0 0 0 1 0 0 0 0 9
##
## Overall Statistics
##
##           Accuracy : 0.9596
##           95% CI : (0.8998, 0.9889)
##           No Information Rate : 0.0909
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.9556
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5
## Sensitivity      1.00000  1.00000  1.00000  1.00000  0.88889  0.88889
## Specificity      1.00000  1.00000  1.00000  1.00000  0.97778  1.00000
## Pos Pred Value   1.00000  1.00000  1.00000  1.00000  0.80000  1.00000
## Neg Pred Value   1.00000  1.00000  1.00000  1.00000  0.98876  0.98901
## Prevalence       0.09091  0.09091  0.09091  0.09091  0.09091  0.09091
## Detection Rate   0.09091  0.09091  0.09091  0.09091  0.08081  0.08081
## Detection Prevalence 0.09091  0.09091  0.09091  0.09091  0.10101  0.08081
## Balanced Accuracy 1.00000  1.00000  1.00000  1.00000  0.93333  0.94444
##
##           Class: 6 Class: 7 Class: 8 Class: 9 Class: 10
## Sensitivity      0.77778  1.00000  1.00000  1.00000  1.00000
## Specificity      0.98889  1.00000  1.00000  1.00000  0.98889
## Pos Pred Value   0.87500  1.00000  1.00000  1.00000  0.90000
## Neg Pred Value   0.97802  1.00000  1.00000  1.00000  1.00000
## Prevalence       0.09091  0.09091  0.09091  0.09091  0.09091
## Detection Rate   0.07071  0.09091  0.09091  0.09091  0.09091
## Detection Prevalence 0.08081  0.09091  0.09091  0.09091  0.10101
## Balanced Accuracy 0.88333  1.00000  1.00000  1.00000  0.99444

```

En general, según los resultados obtenidos k-NN tiene una precisión alta, con la sensibilidad y especificidad cercana a 1 para la mayoría de las clases. En concreto, la mayoría de errores se concentran en la clase 6, ya que tiene todos los valores más pequeños si comparamos con otras clases.

Combinamos todos los datos de ficheros del train a solo uno para obtener la evaluación robusta y aplicamos el método k-nn con caret:

```

all_train <- lapply(1:10, function(i) {
  f <- paste0("vowel-10-", i, "tra.dat")
  x <- read.csv(f, comment.char = "@", header = FALSE)
  names(x) <- c("TT", "SpeakerNumber", "Sex",
               "F0", "F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "F9",
               "Class")

  x
}) %>% bind_rows()

all_train$Class <- factor(all_train$Class)

suppressWarnings({
  knn.model <- train(Class ~ .-Class-TT-SpeakerNumber-Sex,
                    data = all_train,
                    method="knn",
                    metric="Accuracy",
                    tuneGrid = expand.grid(.k = 1:15))
})

knn.model

## k-Nearest Neighbors
##
## 8910 samples
## 13 predictor
## 11 classes: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 8910, 8910, 8910, 8910, 8910, ...
## Resampling results across tuning parameters:
##
##  k  Accuracy  Kappa
##  1  1.0000000  1.0000000
##  2  1.0000000  1.0000000
##  3  0.9997431  0.9997174
##  4  0.9995124  0.9994635
##  5  0.9987263  0.9985986
##  6  0.9977116  0.9974823
##  7  0.9971374  0.9968506
##  8  0.9962794  0.9959066
##  9  0.9957035  0.9952730
## 10  0.9954191  0.9949601
## 11  0.9951228  0.9946341
## 12  0.9948654  0.9943509
## 13  0.9948644  0.9943499
## 14  0.9952066  0.9947264
## 15  0.9950470  0.9945509
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 2.

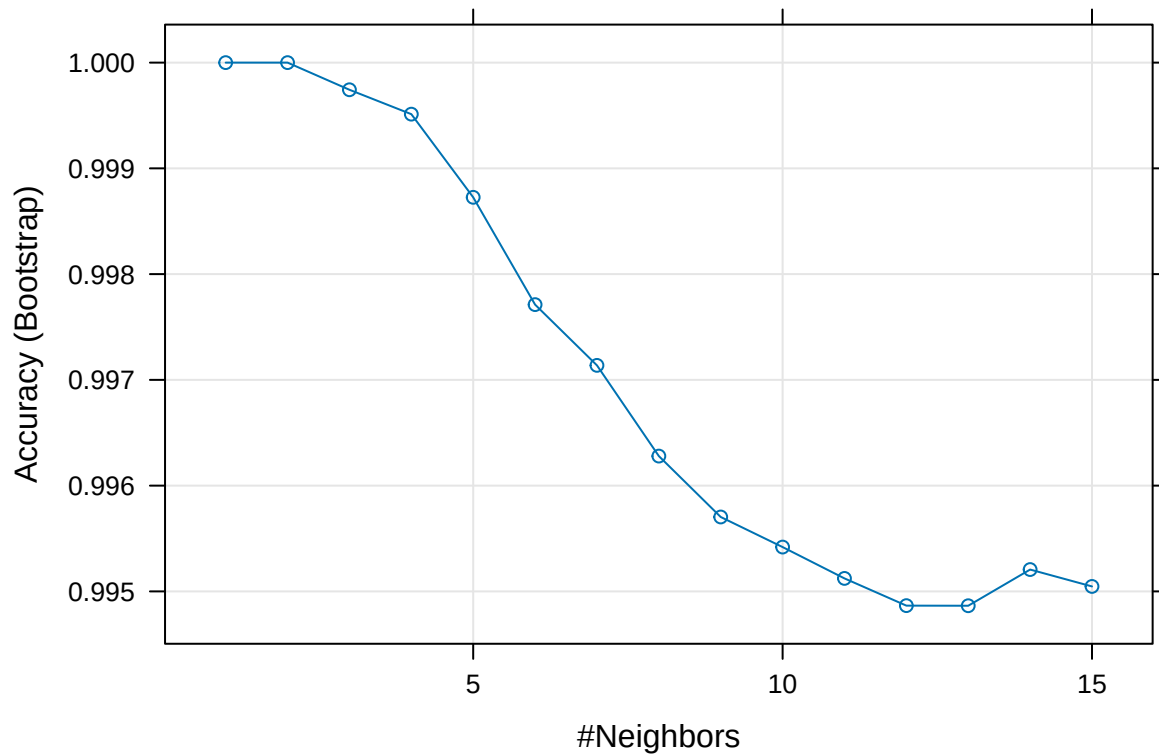
```

Con el bootstrap sobre el conjunto de entrenamiento combinado todos los valores de k entre 1 y 15 muestran una precisión muy alta. Esto indica que el algoritmo k-NN se ajusta muy bien a los datos disponibles, aunque

esta validación está basada en remuestreo del propio conjunto de entrenamiento y, por lo tanto, sobreestima ligeramente el rendimiento real. Los valores de k más adecuados son desde $k = 2$ hasta $k = 5$ que ofrecen un buen compromiso entre alta precisión y un sobreajuste moderado.

Visualización del método k -NN para simplificar la representación de los valores obtenidos:

```
plot(knn.model)
```



Calculamos la medida accuracy para diferentes valores k :

```
k <- c(1, 3, 5, 7, 9, 11)
# Para cada k calculamos la precisión media en el train y el test
get_acc_for_k <- function(k) {
  acc_test <- mean(sapply(1:10, run_knn_fold, k = k, tt = "test"))
  acc_train <- mean(sapply(1:10, run_knn_fold, k = k, tt = "train"))
  c(acc_train = acc_train, acc_test = acc_test)
}

res_mat <- sapply(k, get_acc_for_k)

res_knn <- data.frame(k = k,
  acc_train = res_mat["acc_train", ],
  acc_test = res_mat["acc_test", ])

res_knn

##    k acc_train acc_test
```

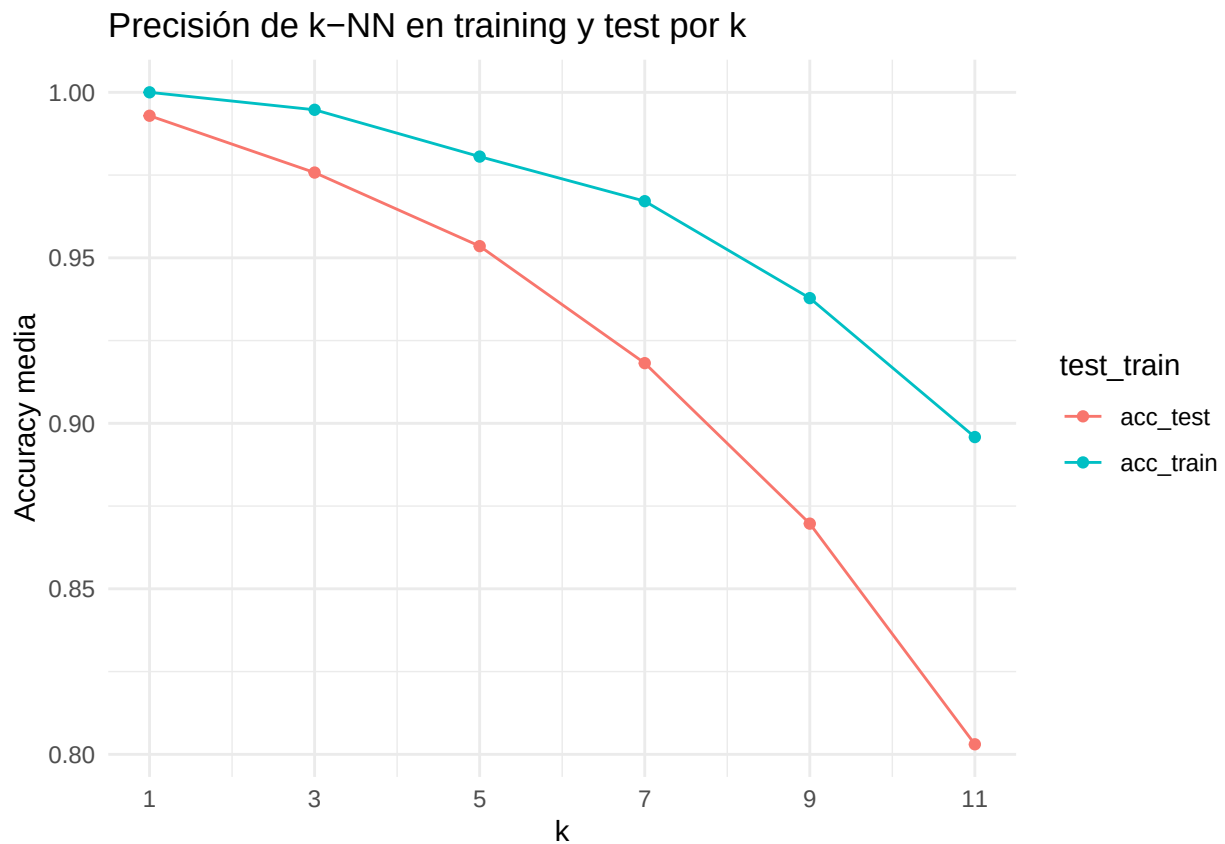


```
## 1  1 1.0000000 0.9929293
## 2  3 0.9947250 0.9757576
## 3  5 0.9805836 0.9535354
## 4  7 0.9671156 0.9181818
## 5  9 0.9378227 0.8696970
## 6 11 0.8958474 0.8030303
```

Visualización de los resultados obtenitos en el último paso:

```
plot_knn <- res_knn %>%
  gather(test_train, accuracy, acc_train, acc_test)

ggplot(plot_knn, aes(x = k, y = accuracy, color = test_train)) +
  geom_line() + geom_point() +
  scale_x_continuous(breaks = k) +
  labs(x = "k", y = "Accuracy media",
       title = "Precisión de k-NN en training y test por k") +
  theme_minimal()
```



Se ve que cuando se aumenta el valor k, disminuyen ambas precisiones y se aumenta la distancia entre el train y test. Por lo tanto, esto indica que k pequeños dan mejor rendimiento.

2. Clasificación con el algoritmo LDA

Definimos una función análoga a run_knn_fold pero para LDA:

```

run_lda_fold <- function(i, tt = "test", return_data = FALSE) {
  file <- paste0("vowel-10-", i, "tra.dat")
  x_tra <- read.csv(file, comment.char="@", header=FALSE)
  file <- paste0("vowel-10-", i, "tst.dat")
  x_tst <- read.csv(file, comment.char="@", header=FALSE)
  names(x_tra) <- c("TT", "SpeakerNumber", "Sex",
                   "F0", "F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "F9",
                   "Class")
  names(x_tst) <- names(x_tra)

  x_tra$Class <- factor(x_tra$Class)
  x_tst$Class <- factor(x_tst$Class)

  # Ajustamos el modelo LDA usando solo el conjunto de train
  model <- lda(Class ~ .-TT-SpeakerNumber-Sex, data = x_tra)

  if (tt == "train") {
    test <- x_tra
  }
  else {
    test <- x_tst
  }

  # Obtenemos las clases predichas y calculamos la precisión
  pred <- predict(model, test)
  mean(pred$class == test$Class)
}

```

Calculamos la precisión media de LDA en todos los folds:

```

acc_lda_test <- mean(sapply(1:10, run_lda_fold, tt = "test"))
acc_lda_train <- mean(sapply(1:10, run_lda_fold, tt = "train"))
res_lda <- data.frame(model = "LDA",
                      acc_train = acc_lda_train,
                      acc_test = acc_lda_test)

res_lda

```

```

##   model acc_train acc_test
## 1   LDA 0.6418631 0.6010101

```

Este análisis muestra que LDA es un clasificador significativamente menos eficiente para conjuntos de datos vowel en comparación con k-NN que tiene los resultados mucho más altos. Entre valores train y test no hay mucha diferencia, por eso este modelo es también bastante estable, aunque tiene menos rendimiento.

La proyección de LD1 y LD2:

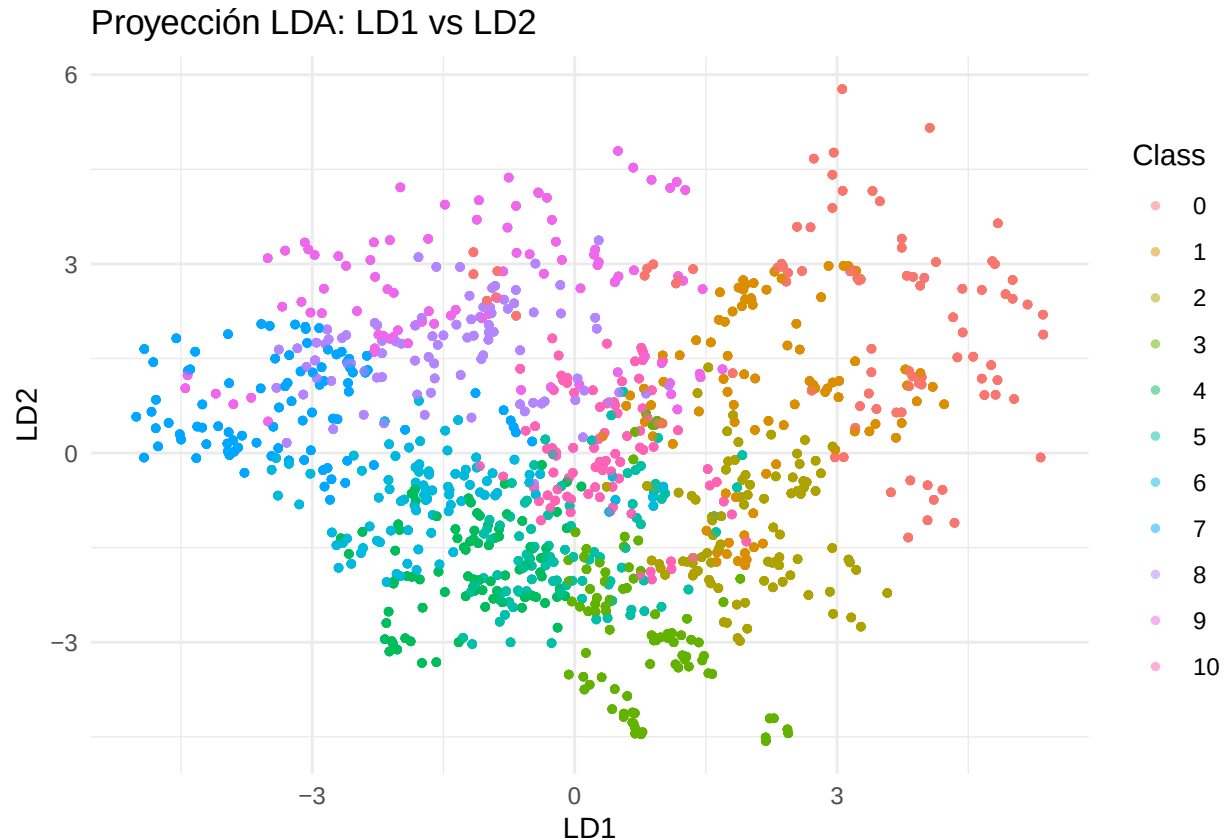
```

lda_model <- lda(Class ~ .-TT-SpeakerNumber-Sex, data = all_train)

lda.data <- cbind(all_train, predict(lda_model)$x)

```

```
ggplot(lda.data, aes(x = LD1, y = LD2, color = Class)) +
  geom_point(alpha = 0.5, size = 1) +
  labs(title = "Proyección LDA: LD1 vs LD2") +
  theme_minimal()
```

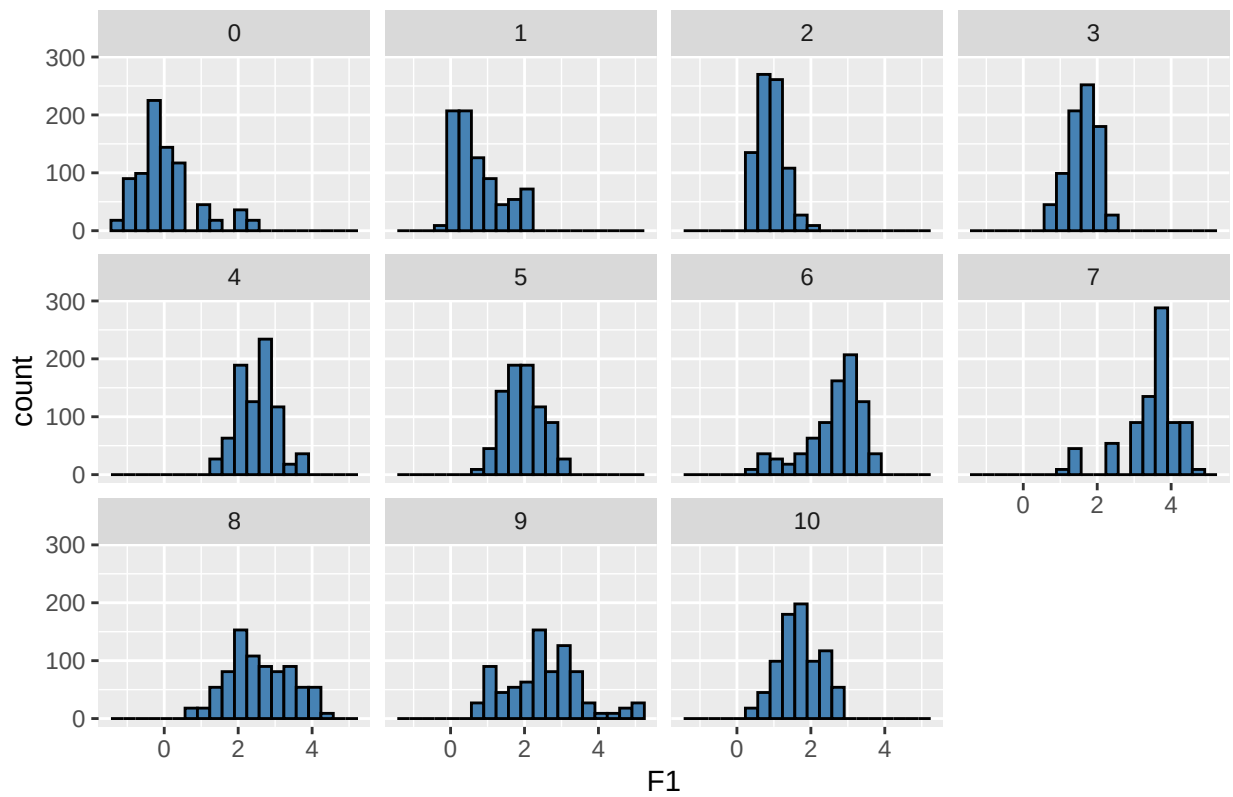


La razón de obtención de los valores de accuracy muy bajos explica por la baja separabilidad de datos que podemos observar gráficamente. Es decir, las clases se solapan fuertemente en el espacio, lo que resulta en la baja precisión observada de LDA.

La observación de la normalidad según los formantes F1 y F2:

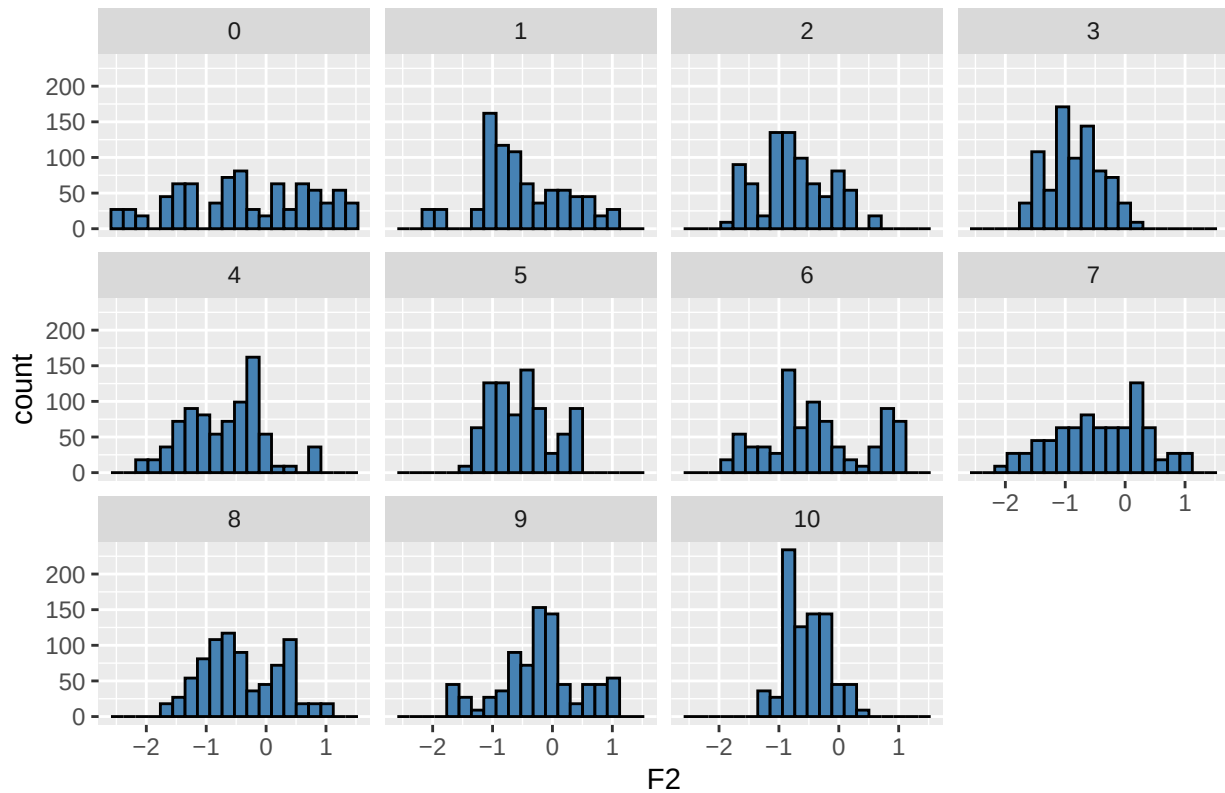
```
ggplot(all_train, aes(x = F1)) +
  geom_histogram(bins = 20, color = "black", fill = "steelblue") +
  facet_wrap(~ Class, ncol = 4) +
  labs(title = "Histogramas de F1 por cada clase")
```

Histogramas de F1 por cada clase



```
ggplot(all_train, aes(x = F2)) +
  geom_histogram(bins = 20, color = "black", fill = "steelblue") +
  facet_wrap(~ Class, ncol = 4) +
  labs(title = "Histogramas de F2 por cada clase")
```

Histogramas de F2 por cada clase



Los histogramas obtenidos son bastante unimodales, aunque tienen asimetría. La normalidad multivariante se cumple solo de forma aproximada. Puesto que la dispersión difiere claramente entre las clases, esto va en contra de la asunción clásica de LDA.

3. Clasificación con el algoritmo QDA

Definimos una función análoga a `run_knn_fold` y `run_lda_fold` pero para QDA:

```
run_qda_fold <- function(i, tt = "test", return_data = FALSE) {
  file <- paste0("vowel-10-", i, "tra.dat")
  x_tra <- read.csv(file, comment.char="@", header=FALSE)
  file <- paste0("vowel-10-", i, "tst.dat")
  x_tst <- read.csv(file, comment.char="@", header=FALSE)
  names(x_tra) <- c("TT", "SpeakerNumber", "Sex",
                    "F0", "F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "F9",
                    "Class")
  names(x_tst) <- names(x_tra)

  x_tra$Class <- factor(x_tra$Class)
  x_tst$Class <- factor(x_tst$Class)

  model <- qda(Class ~ .-TT-SpeakerNumber-Sex, data = x_tra)

  if (tt == "train") {
    test <- x_tra
  }
}
```

```

}
else {
  test <- x_tst
}

pred <- predict(model, test)
mean(pred$class == test$Class)
}

```

Calculamos la precisión media de QDA en todos los folds:

```

acc_qda_test <- mean(sapply(1:10, run_qda_fold, tt = "test"))
acc_qda_train <- mean(sapply(1:10, run_qda_fold, tt = "train"))
res_qda <- data.frame(model = "QDA",
                      acc_train = acc_qda_train,
                      acc_test = acc_qda_test)

res_qda

```

```

##  model acc_train acc_test
## 1   QDA 0.9338945 0.8828283

```

De conformidad con los resultados obtenidos del modelo QDA se puede concluir que tiene mucha más eficiencia comparando con el accuracy de LDA, aunque el método sigue siendo más eficiente.

Comparación de las varianzas de las formantes para tres clases representativas:

```

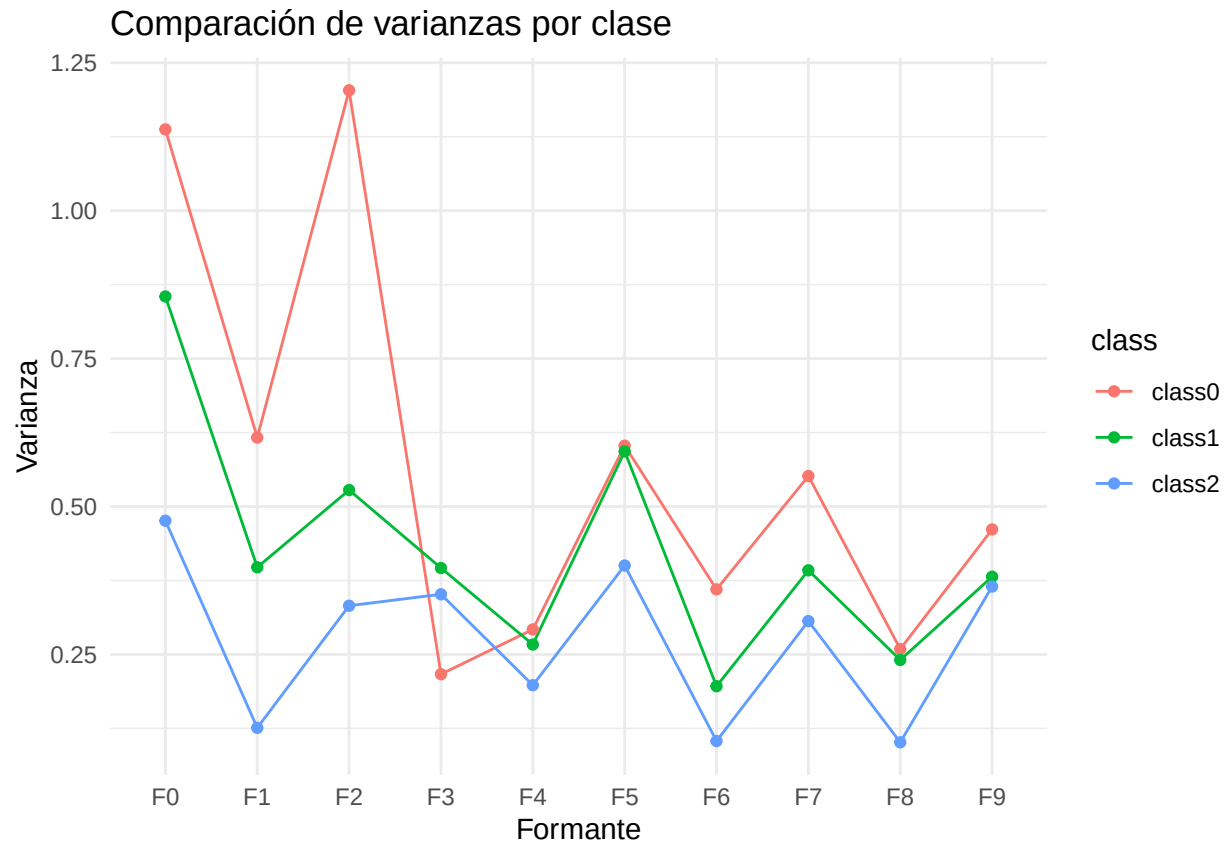
features <- c("F0", "F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "F9")
cov0 <- cov(subset(all_train, Class == 0)[, features])
cov1 <- cov(subset(all_train, Class == 1)[, features])
cov2 <- cov(subset(all_train, Class == 2)[, features])

# calculamos las diagonales de las matrices de covarianza
var_df <- data.frame(feature = features,
                     class0 = diag(cov0),
                     class1 = diag(cov1),
                     class2 = diag(cov2))

var_long <- var_df %>%
  pivot_longer(cols = c(class0, class1, class2),
               names_to = "class",
               values_to = "variance")

ggplot(var_long, aes(x = feature, y = variance, color = class, group = class)) +
  geom_line() +
  geom_point() +
  labs(title = "Comparación de varianzas por clase",
       x = "Formante", y = "Varianza") +
  theme_minimal()

```



El gráfico es una evidencia ilustrativa de que las dispersiones de clases no son iguales. Esto explica definitivamente por qué LDA mostró resultados tan pobres.

4. Comparación de los resultados de tres algoritmos

Comparamos los resultados de los algoritmos k-NN, LDA, QDA construyendo único dataframe:

```
best_k <- 3 # mejor k para k-NN

# Calculamos las precisiones medias de k-NN
acc_knn_train <- mean(sapply(1:10, run_knn_fold, k = best_k, tt = "train"))
acc_knn_test  <- mean(sapply(1:10, run_knn_fold, k = best_k, tt = "test"))

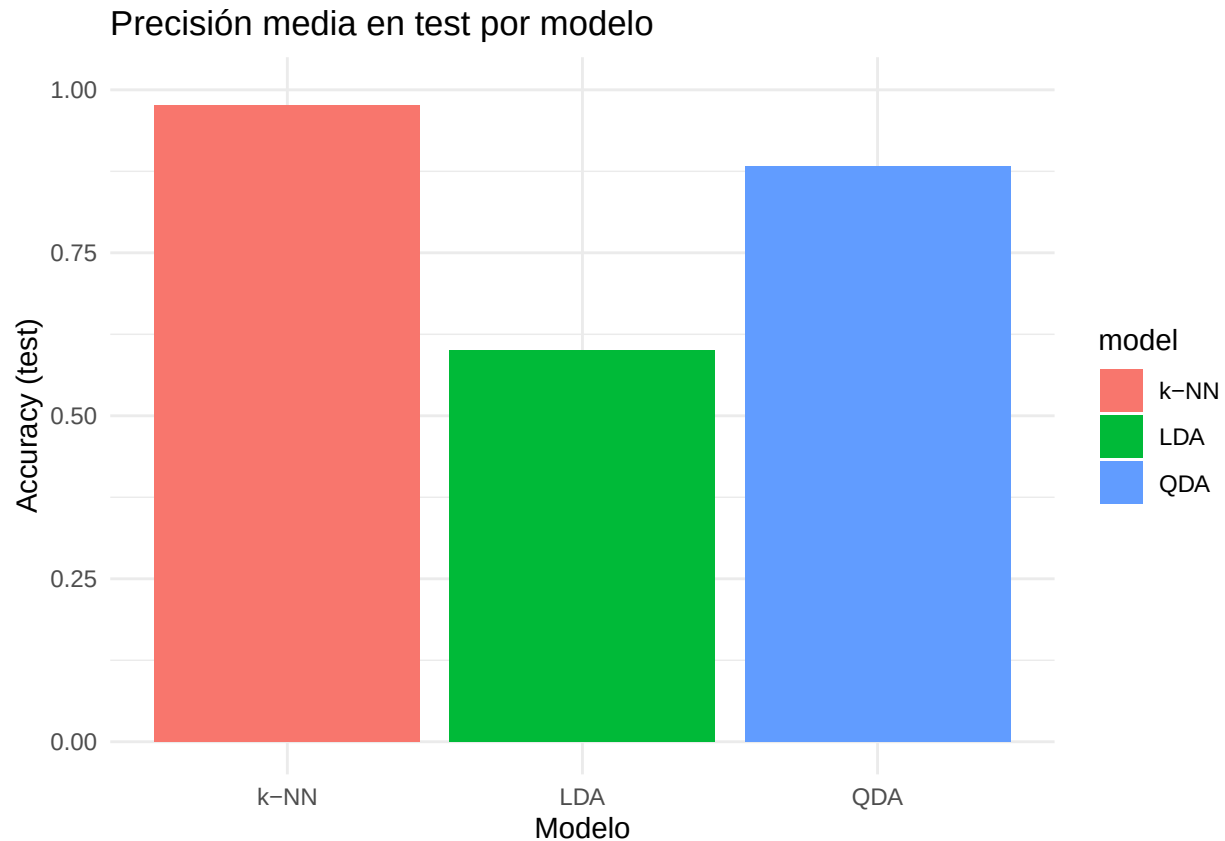
res_all <- rbind(
  data.frame(model = "k-NN", acc_train = acc_knn_train, acc_test = acc_knn_test),
  data.frame(model = "LDA", acc_train = acc_lda_train, acc_test = acc_lda_test),
  data.frame(model = "QDA", acc_train = acc_qda_train, acc_test = acc_qda_test)
)

res_all
```

```
##   model acc_train acc_test
## 1  k-NN 0.9945006 0.9757576
## 2   LDA 0.6418631 0.6010101
## 3   QDA 0.9338945 0.8828283
```

Visualización de esta comparación:

```
ggplot(res_all, aes(x = model, y = acc_test, fill = model)) +
  geom_col() +
  ylim(0, 1) +
  labs(title = "Precisión media en test por modelo",
       x = "Modelo", y = "Accuracy (test)") +
  theme_minimal()
```



El análisis mostró claramente que para la tarea de clasificación del conjunto de datos vowel, el modelo más eficiente es el **k-NN**, logrando una precisión más alta. Este éxito se debe a que las fronteras entre las clases de vocales son complejas y no lineales. Por otro lado, el modelo *LDA* falló estrepitosamente (precisión 60.1%) porque las fronteras no son lineales y que las varianzas son muy distintas. *QDA* mejoró este accuracy, pero no alcanzó el rendimiento de k-NN. En consecuencia, el modelo final recomendado para esta tarea es el **k-NN** con **k = 3**.